Deadline Coach

A Telegram assistant that collects my deadlines from Moodle and from my own messages, reminds me before they are due, warns me when a teacher moves a date, and plans my week.

Author	Saty Viard Laroque, AIDAMS 3A	**Tool**	n8n Cloud
Channel	Telegram bot	**Database**	Supabase (Postgres)
Sources	Moodle calendar (iCal), Telegram	**AI**	OpenRouter, free models
Level	**3** (5 workflows)	**Date**	October 2026
Code	github.com/exilonn/deadline-coach (workflows, Docker setup, README)		

1 The need

Who has the problem. Students following six or seven courses at once. Each course has its own Moodle page, and some deadlines are only announced in class, so they are easy to lose track of.

What we do today instead. We open each Moodle course one by one and note the dates by hand. Moodle has a calendar, but nobody checks it every day, it does not warn you when a date changes, and it only contains the deadlines that teachers created as Moodle activities.

What the bot changes. The deadlines that are on Moodle arrive by themselves. The others I add with one message, in my own words. The bot then reminds me 7, 3 and 1 day before each deadline, tells me when a teacher moves a date, and on `/plan` proposes how to split the work over the week.

2 Triggers and how I test them

Trigger	What it starts
Telegram Trigger	Every message to the bot: `/add`, `/deadlines`, `/done`, `/plan`, `/sync`, `/help`, or anything else.
Schedule, 08:00 daily	The morning reminder digest.
Schedule, every 6 hours	The Moodle sync. `/sync` starts the same workflow on demand.
Error Trigger	Any failure in the other workflows.

Each schedule has a **Manual Trigger** next to it, so I can run it immediately instead of waiting. I kept a fixed set of test inputs and resent them after every change:

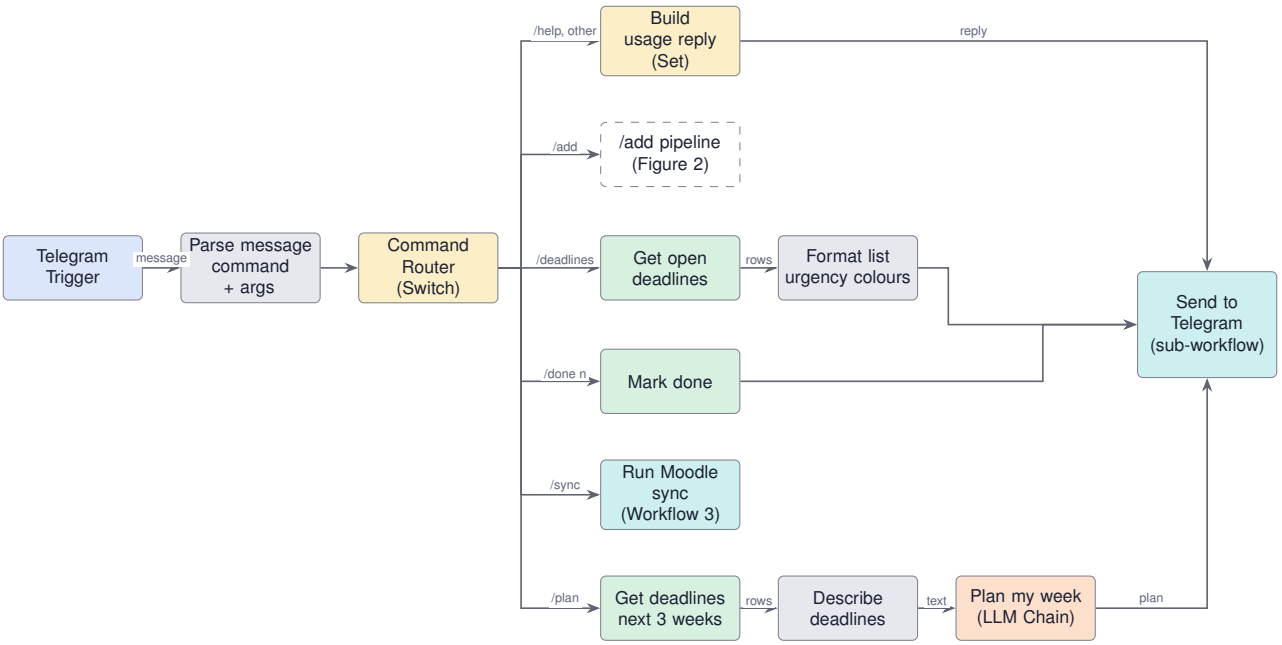
Test input	Expected behaviour
`/add CV final report 28/10 20:00`	AI extracts the deadline, it is saved, the bot confirms with its number.
`/add env econ paper due next monday`	“next monday” becomes a real date; “env econ” becomes Environmental Economics.
The same `/add` twice	Second time: “Already saved as #N”, nothing inserted.
Same title, new date	“moved to . . . ”, the date is updated and the reminders start again.
`/add blabla`	“I couldn’t find a deadline”, nothing saved.
`/dealines, hello`	“Did you mean /deadlines?” or the usage message.
Reminders run twice	Each reminder is sent once; the second run sends nothing.
Moodle sync run twice	First run inserts the events, second run changes nothing.
A node failing in a published workflow	Telegram alert with the workflow, the node and the error.

3 Architecture

The project is split into five workflows, each with one job. Every message the bot sends goes through the same sub-workflow. The labels on the arrows show what travels between blocks.

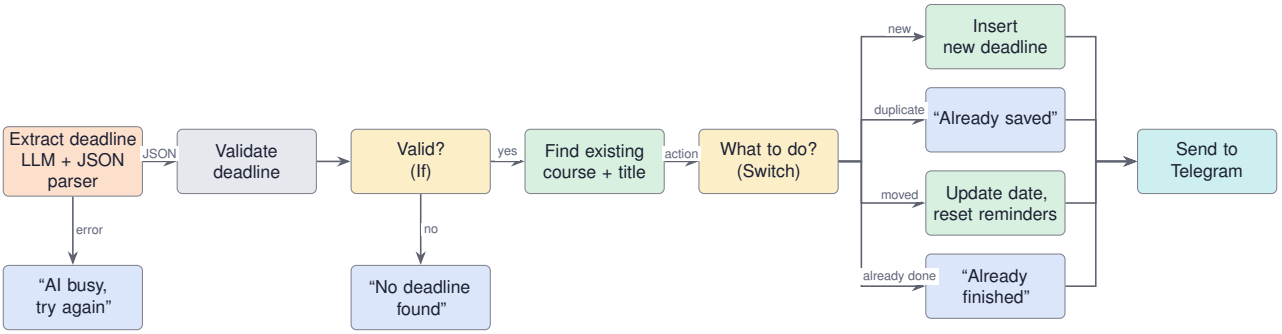
Trigger / reply | Switch, If, Set | Code (Python) | External API | AI | Database | Sub-workflow | Error handling

Workflow 1: Main (receiving a message)



Both AI nodes (*Extract deadline* and *Plan my week*) also have an error output that replies “The AI is busy, try again” (drawn in Figure 2 only).

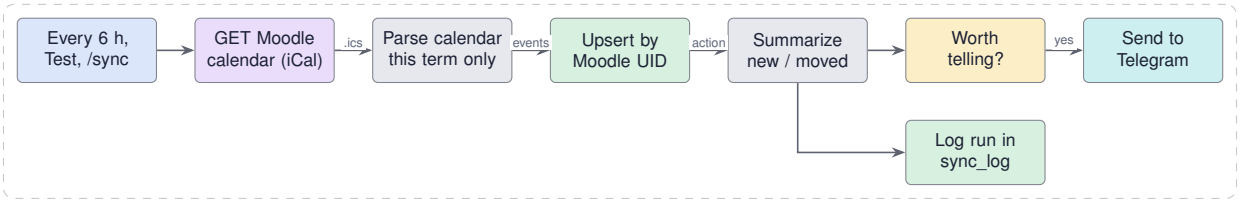
The /add pipeline (inside Main)



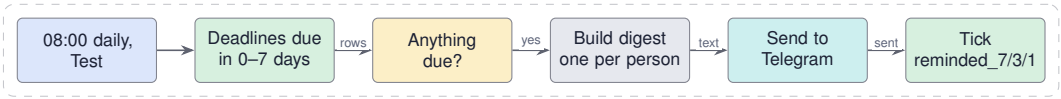
The AI must answer in a fixed JSON shape (found, course, title, due, weight). The Code node checks that the date really exists before anything is saved, because the AI’s output is never trusted as it is. “No deadline found” and “AI busy” also go through Send to Telegram (arrows not drawn).

Workflows 2 and 3: morning reminders and Moodle sync

Workflow 3: Moodle sync



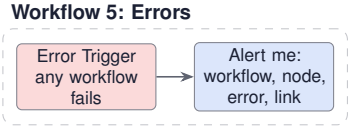
Workflow 2: Morning reminders



Workflows 4 and 5: Send to Telegram and error handling

Workflow 4: Send to Telegram

Workflow 5: Errors



Database. One Supabase (Postgres) table `deadlines`: course, title, due date, grade weight, status (open or done), source (manual or moodle), the Moodle event ID (unique), and three flags `reminded_7`, `reminded_3` and `reminded_1`. A second table `sync_log` records each Moodle sync. Dates are stored in UTC and converted to Paris time in the SQL, so the Python code never deals with time zones.

4 Design choices

- **One job per workflow.** Receiving messages, reminders, syncing and error handling run on different triggers and fail independently. A broken Moodle feed cannot stop the bot from answering.
- **One sender for every message.** All replies go through the *Send to Telegram* sub-workflow, which splits messages above Telegram's 4,096-character limit and retries. The other workflows only say "send this text to this chat". Only the error alert calls Telegram directly, so it still works if the sub-workflow itself is broken.
- **Never trust the AI blindly.** The AI is used for language (reading "next monday", writing a plan), never for storage decisions. Its output must match a JSON schema, then a Code node checks the date exists.
- **Idempotent sync and reminders.** The sync matches events on Moodle's unique ID, so running it twice changes nothing. Reminder flags are only ticked after Telegram confirms the message, so a failed send is retried the next day instead of being lost.
- **Ask the database to do the date maths.** Days left, Paris time and readable dates come from SQL, which keeps the Code nodes short and readable.

5 Error handling and retries

- **Retry on fail** (3 tries) on both AI nodes, the Moodle download, the Telegram sender, and the database writes of the Moodle sync and the reminders.
- **AI failures:** free models are often rate-limited. If the retries still fail, the AI node's error output sends the user "The AI is busy, try again in a minute" instead of leaving them without an answer.
- **Bad input:** unknown commands get the usage message, typos get "Did you mean . . . ?", and messages without a date are refused before reaching the database.
- **Error workflow:** any other failure sends me a Telegram alert with the workflow, the failing node, the error message and a link to the execution. It proved useful for real: on 22 October at 08:00 the reminder digest failed (a chat ID read as text instead of a number). The alert pointed straight to the failing node and I fixed it the same morning. Because reminder flags are only ticked after a successful send, no reminder was lost.

6 Minimum technical requirements

Requirement	Where it is in the project
If or Set node	If: *Valid?*, *Anything due?*, *Worth telling?*. Set: *Build usage reply*.
Code node	Python: Parse message, Validate deadline, Format list, Describe deadlines, Build digest, Parse calendar, Summarize sync, Split long message.
Router or If/Switch	*Command Router* (7 routes) and *What to do?* (new / duplicate / moved / already done).
API connection	HTTP Request to the Moodle iCal export; OpenRouter API; Telegram Bot API; Supabase.
Message trigger	Telegram Trigger.
Database	Supabase (Postgres): `deadlines` and `sync_log`.
AI node	Two LLM Chains: *Extract deadline* (structured JSON output) and *Plan my week*.
Sub-workflow	*Send to Telegram*, called by Main, Reminders and Moodle Sync; *Moodle sync*, called by `/sync`.
Error handling and retries	Retry on fail on the AI, HTTP and Telegram calls and the key database writes; error output on both AI nodes; Error Workflow.

7 Level reached, and what changed since the proposal

Level 3. The system keeps state between runs (reminder flags, Moodle event IDs), detects duplicates and date changes, uses sub-workflows and an error workflow, and retries the calls most likely to fail.

Cut line (decided before building). If time had run short, I would have dropped, in this order: `/plan`; then the 7/3/1 reminder tiers (keeping a single reminder 2 days before); then the automatic Moodle sync (keeping `/add` only). Never

cut: `/add` with AI extraction, `/deadlines`, the daily reminder and the usage message. In the end nothing had to be cut.

- **AI provider:** OpenRouter free models instead of Groq, as in the course lab.
- **Sub-workflow:** instead of *Parse & Save*, the shared sub-workflow is *Send to Telegram*. It is used by every workflow that replies to the user, which made it the more useful one to extract. Duplicate detection stayed in Main.
- **Moodle sync:** one SQL "upsert" returns new, moved or unchanged for each event, instead of a separate Switch. A summary message is sent only when something changed.
- **`/plan`:** built as an LLM Chain that receives the deadlines as text, rather than an AI Agent with a database tool. It is simpler and the data the AI sees is always exactly what the database holds.
- **Found while building:** the Moodle calendar only contains deadlines created as Moodle activities. Many deadlines are only announced in class or in course documents, so `/add` remains essential.

8 What would change on WhatsApp

The design would stay the same; only the two ends change. The Telegram Trigger would become the WhatsApp Trigger, fed by a Meta Cloud API webhook that has to be verified, and *Send to Telegram* would become *Send to WhatsApp* using the WhatsApp Business Cloud node. Because every message already goes through that one sub-workflow, only one workflow would change. Two WhatsApp rules affect this project: the bot can only message registered numbers, and outside 24 hours after the user's last message it can only send pre-approved templates. The 08:00 reminder would therefore have to be an approved template, while replies to commands would work as they do now.

9 Limits and possible improvements

- Duplicate detection matches the exact course and title, so "Final report" and "Final project report" count as two deadlines. Fuzzy matching would fix this.
- A deadline added by hand can also arrive later from Moodle, giving two rows.
- Deadlines that are not on Moodle need `/add`. A next step would be to forward a document to the bot and let the AI extract every deadline in it.
- Free AI models are slow and rate-limited. A paid model would make `/add` and `/plan` faster and more reliable.

Use of AI. I used an AI assistant to help design the workflows and to draft and review the Code nodes and SQL queries. I built, tested and adjusted every node myself in n8n, and I can explain each of them.